Public Key Crypto notes - 01

By contributors

February 12. 2026.

1 Symmetric key crypto recap

- Symmetric key algorithms, schemes (AES, DES, block ciphers, MACs, etc.)
- Given two parties A, B
- They have a *shared secret key* k
- There is an encryption function $Enc_k(m) = c$
- And there's also a decryption function $Dec_k(c) = m$
- Properties:
  - Symmetric: the roles, keys of A and B are symmetric
  - Fast and efficient encryption & decryption

2 Public key (asymmetric) cryptography (PKC)

- Encryptions (RSA, ElGamal, elliptic curve based cryptography)
- Digital signatures
- Syntax of PKC:
  - There are two kinds of keys
  - e - Encryption key, which is public
  - d - Decryption key, which is private, kept in secret
- $c = Enc_e(m)$
- $m = Dec_d(c)$

2.1 Properties of PKC

- Asymmetric
- Slower, computationally more expensive calculations compared to symmetric key cryptography

- Since it is so expensive to individually encrypt and decrypt each message in an asymmetrical way, modern ciphersuites often do the following:
  - Exchange the shared secret key between two parties using each party's keypairs (e.g. using the Diffie-Hellman key exchange),
  - Once the shared key is exchanged in an encrypted way over any channel, continue the communication using a symmetric cipher (like AES in GCM mode) and the shared key (which is the same for both parties).
- Ralph Merkle: "The inventor of PKC" and commutative encryption.

Definition 2.1 (Commutative encryption)**.**

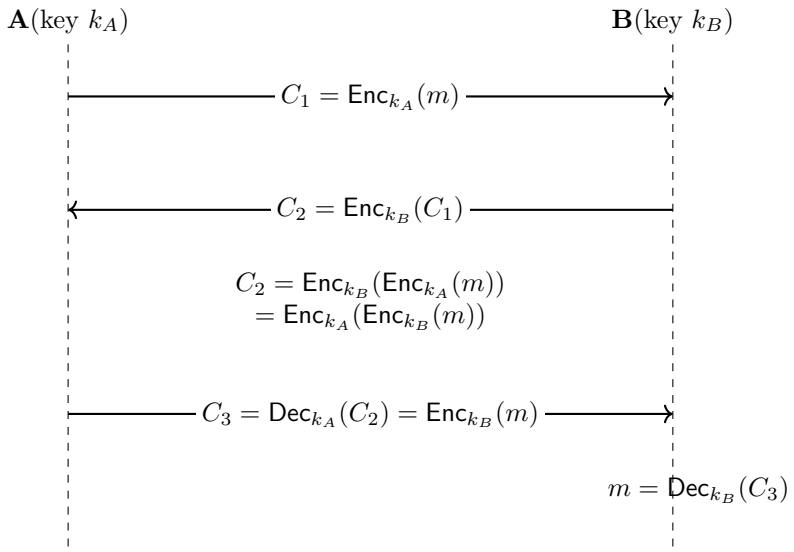
An encryption scheme $\mathsf{Enc}_k(\cdot)$ is *commutative* if for all messages m and all keys k_1, k_2,

$$\mathsf{Enc}_{k_1}(\mathsf{Enc}_{k_2}(m)) = \mathsf{Enc}_{k_2}(\mathsf{Enc}_{k_1}(m)).$$

Equivalently, the order of encryption under different keys does not matter:

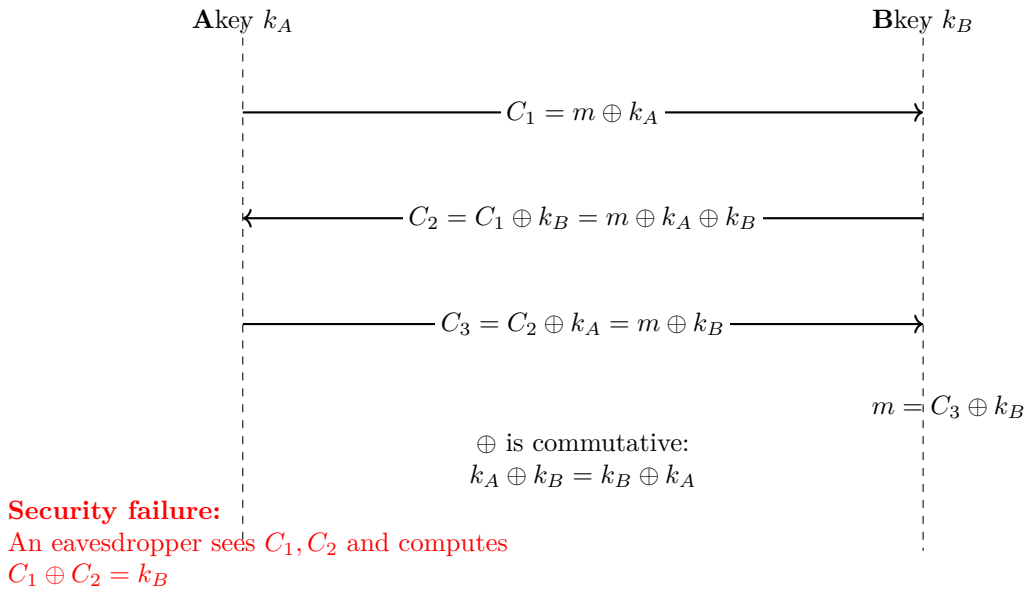
$$\mathsf{Enc}_{k_1} \circ \mathsf{Enc}_{k_2} = \mathsf{Enc}_{k_2} \circ \mathsf{Enc}_{k_1}.$$

2.2 R. Merkle - exchanging a shared key using a commutative encryption scheme



- This way, the shared key never gets sent on the insecure channel as a plaintext.
- It always travels with at least one layer of encryption.
- Let's see if the one-time pad scheme could be used this way.

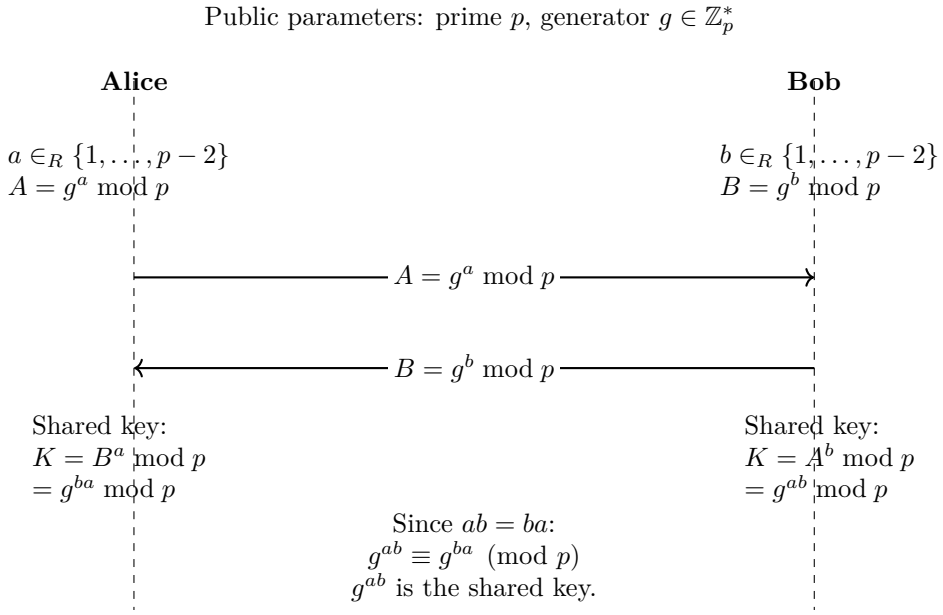
One-Time Pad as Commutative Encryption (Broken Example)



- The OTP scheme could not be used this way, since B's key would leak in the presence of an eavesdropper.

2.3 Diffie-Hellman (DH) key exchange

- Whitfeld Diffie and Martin Hellman developed a secure commutative encryption scheme in 1973 and is mainly used as a key exchange protocol.
- Take a large prime p and a number $g \pmod p$, where $g \in \mathbb{Z}_p$



- If a can be computed from g^a. then this key exchange protocol would be breakable.

- This problem is the *Discrete logarithm problem*, which is computationally hard.
- Questions:
- Is the Discrete logarithm problem really hard?
- What is the generator number g? How can we choose it?
- How large would p need to be?

3 Number theoretic backgrounds

Definition 3.1 (Order of the generator)**.** Let p be a prime number and $g \in \mathbb{Z}_p, g \neq 0$. Then, the order of g ($ord_p g$) is the smallest $k \leq 1$, such that:

$$g^k \equiv 1 \pmod{p}$$

Remark 3.1. This is well-defined by Fermat's little theorem: $g^{p-1} \equiv 1 \pmod{p}$.

Remark 3.2. $ord_p g$: The order of g is the number of possible powers of g modulo p.

$$g^0 = 1; g^1 = g; g^2; g^3; \dots g^{ord_p(g)-1} \tag{1}$$
$$g^{ord_p(g)} = 1; g^{ord_p(g)+1} = 1 \cdot g = g \tag{2}$$

Claim 3.1. $ord_p(g)|p-1$ ($ord_p(g)$ divides $p-1$)

Proof. Let $\sigma = ord_p(g)$

$$p - 1 = \sigma \cdot g + r \qquad 0 \leq r \leq \sigma \tag{3}$$
$$1 = g^{p-1} = g^{\sigma \cdot g + r} = \qquad \text{(by Fermat's little theorem)} \tag{4}$$
$$= (g^\sigma)^g \cdot g^r \qquad \text{(where } g^\sigma \text{ comes from the definition of the order of } g) \tag{5}$$

i.e.: $1 = g^r$ by the minimality of the ord, $r = 0$ □

Definition 3.2 (generator)**.** Let p be a prime and $g \in \mathbb{Z}_p, g \neq 0$. Then g is a generator if:

$$ord_p(g) = p - 1$$

Remark 3.3. One can prove that there are always generators.

Theorem 3.1. Let p be a prime and write $p - 1 = q_1^{e_1} \cdot q_2^{e_2} \cdot \dots \cdot q_t^{e_t}$. Then g is a generator if and only if:

$$g^{\frac{p-1}{q_i}} \equiv 1 \pmod{p}$$

for all i.

Proof. Prove from left-to-right: We know that g is a generator. Then:

$$ord_p(g) = p - 1$$

So, by the minimality of the ord, $g^{\frac{p-1}{q_i}} \not\equiv 1 \pmod{p}$.

Prove from right-to-left: Suppose that g is *not* a generator, i.e.: $g^k \equiv 1 \pmod{p}$, $1 \leq k < p-1$. As $k|p-1$, write $p - 1 = s \cdot k$. Let $q_i|s$, then:

$$g^{\frac{p-1}{q_i}} = g^{\frac{s \cdot k}{q_i}} = g^{\frac{s}{q_i} \cdot k} = (g^k)^{\frac{s}{q}} = 1$$

□

Remark 3.4. Typical choice for p is $p = 2q + 1$, where q is a prime. Here, p is called a "safe" prime, while q is called a "Sophie Germain prime".[1]

[1] https://en.wikipedia.org/wiki/Safe_and_Sophie_Germain_primes

Public Key Crypto notes - 02

By contributors

February 16. 2026.

1 Diffie-Hellman key exchange recap

- Given: p prime, $g \in \mathbb{Z}_p$ generator with large order $q = ord_p(g)$
- Randomly chosen $a, b \to g^a, g^b \to g^{ab}$ is the shared key
- Detailed description can be found in the previous lecture's notes.
- Typically, the generator g is chosen so that $q = ord_p(g)$ is a large prime (i.e. $q \approx c \times p$, where $c = 2, 3, 5, \ldots$ is typically a small constant)

Remark 1.1. As $g^q \equiv 1 \pmod{p}$, the exponents a, b can be computed on modulo q. Because of this, using $\mathbb{Z}_q$ is also a correct notation.

2 How to break this protocol?

Definition 2.1 (Discrete logarithm)**.** Let p be a prime and let $g \in \mathbb{Z}_p^*$ (the multiplicative group modulo p), typically with g a generator of the group. For an element $y \in \mathbb{Z}_p^*$, the discrete logarithm of y to the base g modulo p is the integer x such that

$$g^x \equiv y \pmod{p},$$

where $0 \leq x \leq p-2$. We write this as

$$x = \log_g y \pmod{p}.$$

If g is a generator of $\mathbb{Z}_p^*$, then such an x exists and is unique modulo $p-1$.

Remark 2.1. In these notes, when we talk about log, we are usually in base-2 logarithm ($\log_2$).

Claim 2.1. For a given $h = g^a$, computing $a = \log_q(h)$ is computationally hard.

2.1 Attacks against the Diffie-Hellman key exchange

2.1.1 Brute-force

- Running time: Size of the set of the possible expontents. $\mathcal{O}(q) = \mathcal{O}(p)$
- Given p, q with order $q = ord_p(g)$ and $h = g^a$, try all $a' \in \mathbb{Z}_q^*$ values and check whether $g^{a'} = h$
- Space complexity: $\mathcal{O}(1)$
- This attack eventually succeeds, given enough time.

2.1.2 Baby-step-giant-step algorithm

Given p, q, where $q = ord_p(g)$, and $h = g^a$, let $a = i \times m + j$ with $m = \lceil \sqrt{q} \rceil, 0 \leq i, j < m$. Then:

$$h = g^a = g^{i \times m + j}, \tag{1}$$
$$g^j = h \times (g^{-m})^i. \tag{2}$$

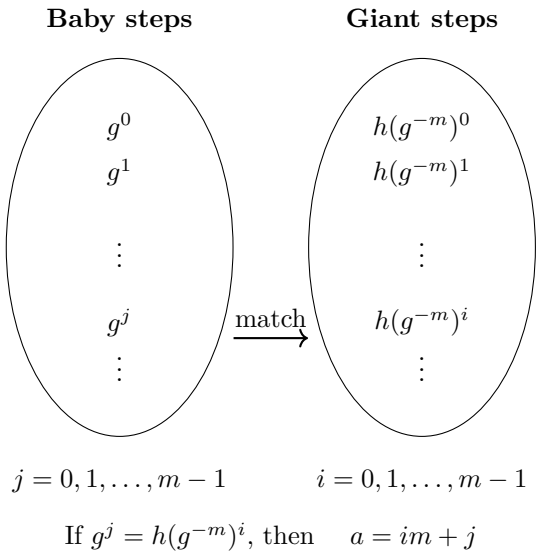


Figure 1: Baby-step–giant-step algorithm: matching baby steps and giant steps to recover a.

- Running time: $\mathcal{O}(m) = \mathcal{O}(\sqrt{q}) = \mathcal{O}(\sqrt{p})$
- Space complexity: Store both sets: $\mathcal{O}(m) = \mathcal{O}(\sqrt{q}) = \mathcal{O}(\sqrt{p})$

Remark 2.2. There are other algorithms like Pollard's rho (ρ) algorithm with time complexity $\mathcal{O}(\sqrt{p})$ and with $\mathcal{O}(1)$ space complexity.

Before presenting one last attack, we have to talk about key sizes (how large the prime p should be).

2.1.3 Key size

Let's answer the question of how large should the prime p be to achieve the same security level as AES-128, i.e. breaking would need $\approx 2^{128}$ time.

- Brute-force: $\mathcal{O}(p) = \mathcal{O}(2^{\log_2 p}) \rightarrow \log p \approx 128$
- Baby-step-giant-step: $\mathcal{O}(\sqrt{p}) = \mathcal{O}(2^{\frac{1}{2}\log p}) \rightarrow \log p \approx 256$

With this clarification, we can move onto the next attack.

2.1.4 Index calculus algorithm

- We won't give the precise details of the algorithm, only that it is the most efficient one for trying to find discrete logarithms.
- Running time: $\mathcal{O}(e^{(\sqrt[3]{\frac{64}{9}}+\epsilon)(\log p)^{\frac{1}{3}}(\log\log p)^{\frac{2}{3}}})$
- $\mathcal{O}(e^{(\sqrt[3]{\frac{64}{9}}+\epsilon)(\log p)^{\frac{1}{3}}(\log\log p)^{\frac{2}{3}}}) \to 2^{128} \to \log p \approx 3072$

Remark 2.3. This is called a *subexponential* running time.

- Exponential time: $\mathcal{O}(2^n)$
- Polynomial time: $\mathcal{O}(n^c)$
- $\mathcal{O}(n^c) < \mathcal{O}(e^{c \cdot n^{\frac{1}{3}}}) < \mathcal{O}(2^n)$

Definition 2.2 (Hardness of the discrete logarithm problem)**.** We say that the discrete logarithm problem is computationally hard if for all probabilistic polynomial time (PPT) adversary A, input size n (i.e. $n = \log p$) and negligible function $negl(n)$:

$$Pr[A(g^a) = a] \leq negl(n)$$

Definition 2.3 (Computational Diffie-Hellman Problem)**.** Given

$$(g, g^a, g^b),$$

for a randomly chosen generator g and random

$$a, b \in \{0, \ldots, q-1\},$$

it is computationally intractable to compute the value

$$g^{ab}.$$

Definition 2.4 (Hardness of the Computational Diffie–Hellman Problem)**.** We say that the Computational Diffie–Hellman problem (CDH) is hard if, for all security parameters n (i.e., $n = \log q$) and for all PPT adversaries A,

$$\Pr\left[A(g, g^a, g^b) = g^{ab}\right] \leq \text{negl}(n),$$

where g is a generator of a cyclic group G of order q, and $a, b \in_R \{0, \ldots, q-1\}$.

Definition 2.5 (Hardness of the Decisional Diffie–Hellman Problem)**.** We say that the Decisional Diffie–Hellman problem (DDH) is hard if there exists no PPT adversary that can distinguish g^{ab} from a random group element.

Formally, DDH is hard if, for all security parameters n and all PPT adversaries A,

$$\left|\Pr\left[A(g, g^a, g^b, g^{ab}) = 1\right] - \Pr\left[A(g, g^a, g^b, g^c) = 1\right]\right| \leq \text{negl}(n),$$

where $a, b, c \in_R \{0, \ldots, q-1\}$.

2.1.5 Additional remarks

Suppose that an adversary A is given p, q, g. For given r, s, z, it outputs

$$A(r,s,z) = \begin{cases} 1 & \text{if } z = g^{\log r \cdot \log s}, \\ 0 & \text{otherwise.} \end{cases}$$

Remark 2.4. If the discrete logarithm problem is hard, that also means that the CDH problem is hard and vice-versa. There are also cases where the CDH problem is supposed to be hard, but the DDH problem is easy.

Remark 2.5. One can define the security of key exchange protocols and show that if the DDH problem is hard, then the Diffie-Hellman key exchange protocol is secure. We will make this fact clearer when we are discussing encryption protocols

3 Introduction to public key encryption schemes

Definition 3.1 (Public key encryption scheme)**.**

A Public Key Encryption (PKE) scheme is a triple $\Pi = (Gen, Enc, Dec)$, where:

1. Gen: Input: security parameter 1^n (in other words, an n length string of 1s), output: (pk, sk) (public key, secret key)
2. Enc: Input: pk, m, output: $c \leftarrow Enc_{pk}(m)$, where $\leftarrow$ denotes that Enc is not necessarily deterministic
3. Dec: Input: sk, c, output: m or $\perp$, where $\perp$ is an error indicating an invalid ciphertext

We require that — except for a negligible probability — if $(pk, sk) \leftarrow Gen(1^n)$ and $c \leftarrow Enc_{pk}(m)$, then $Dec_{sk}(c) = m$.

Public Key Crypto notes - 03

By contributors

February 23. 2026

1 Public Key Encryption recap

Definition 1.1 (Public key encryption scheme)**.**

A Public Key Encryption (PKE) scheme is a triple $\Pi = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$, where:

1. *Gen*: Input: security parameter 1^n (in other words, an n length string of 1s), output: (pk, sk) (public key, secret key)
2. *Enc*: Input: pk, m, output: $c \leftarrow \mathsf{Enc}_{pk}(m)$, where $\leftarrow$ denotes that Enc is not necessarily deterministic
3. *Dec*: Input: sk, c, output: m or $\perp$, where $\perp$ is an error indicating an invalid ciphertext

2 Security of a Public Key Encryption scheme

- Model: The adversary A is a PPT algorithm and A knows all the public parameters (the $\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}$ algorithms and the pk public key)
- The adversary A eavesdrops ciphertext c
- Goal of A is to reveal *any* information on message m
- If this goal is accomplished by A, the scheme is considered broken

Remark 2.1. This is called the CPA (Chosen-Plaintext Attack) model.

Definition 2.1 (CPA experiment)**.**

1. Run the key generation algorithm $\mathsf{Gen}(1^n)$, obtaining the (pk, sk) pair.
2. A is given pk, and outputs m_0, m_1 messages.
3. Choose a random bit $b \in_R \{0, 1\}$ and $c \leftarrow Enc_{pk}(m_b)$.
4. A is given c, and outputs the guess b'.
5. If the guess $b' = b$, we say that A has succeeded and $PubK^{cpa}_{A,\Pi}(n) = 1$, otherwise, $PubK^{cpa}_{A,\Pi}(n) = 0$

Definition 2.2 (CPA-secure encryption scheme)**.** The $\Pi = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ public key encryption scheme is CPA-secure if:

$$\Pr[PubK^{cpa}_{A,\Pi}(n) = 1] \leq \frac{1}{2} + \mathsf{negl}(n)$$

3 ElGamal encryption scheme

An "offline version" of the Diffie-Hellman key exchange protocol.

Definition 3.1 (ElGamal encryption scheme)**.**

- Gen:
  - Input: 1^n
  - Output: Prime $p, q \in \mathbb{Z}_p, q$ (the order of g, which is typically a prime), $x, h = g^x$.
  - In a shorter way: $p, q, g, x, h = g^x$
  - $pk = (p, q, g, h)$
  - $sk = x$
- Enc:
  - Input: $pk, m = g^i, i = 0, \ldots, q-1 =< g >$ (alternate notation)
  - Output: $c = (g^y, m \cdot h^y)$, with $y \in_R \mathbb{Z}_q$ (y is also referred to here as the *session key*)
- Dec:
  - Input: $sk, c = (c_1, c_2)$
  - Output: $\frac{c_2}{c_1^x}$

Correctness: $\frac{c_2}{c_1^x} = \frac{m \cdot h^y}{(g^y)^x} = m \cdot \frac{g^{xy}}{g^{xy}} = m$

Theorem 3.1 (CPA-security of the ElGamal public key encryption scheme)**.** If the Decisional Diffie-Hellman (DDH) problem is hard, then the ElGamal public key encryption scheme is CPA-secure.

Recall 3.1. DDH: Given g^x, g^y, h_3, decide whether $h_3 = g^{xy}$

Proof. Our goal is to prove that $\forall PPT$ adversaries A:

$$\Pr[PubK_{A,\Pi}(n) = 1] = \frac{1}{2} + \mathsf{negl}(n)$$

Assume towards contraction that we have an adversary A. We define a distinguisher D which tries to solve the DDH problem.

We define a fake encryption scheme Π':

- Input: $m, pk = (p, q, g, h)$
- Output: $c = (g^y, m \cdot g^z)$, with $y, z \in_R \mathbb{Z}_q$
- There is no Dec algorithm defined

We define a distinguisher D:

- Input: $h_1 = g^x, h_2 = g^y m h_3$
- D tries to determine whether $g^{xy} = h_3$
- The algorithm is as follows:

1. Under pk, run A to obtain two messages: m_0, m_1
2. Choose a random bit $b \in_R \{0,1\}$
3. Calculate $c_1 = h_2, c_2 = m_b \cdot h_3$
4. Give (c_1, c_2) to A, obtain a guess b'

- Output:

$$D = \begin{cases} 1 & \text{if } b = b', \\ 0 & \text{otherwise.} \end{cases}$$

We analyse D as follows:

- Two cases:

1. If $h_3 = g^z, z \in_R \mathbb{Z}_q$, then $(c_1, c_2) = (g^y, m_b \cdot g^z)$, which means that this is the fake encryption scheme Π'
   - $\Pr[D(g^x, g^y, g^z) = 1] = \Pr[PubK^{cpa}_{A,\Pi'}(n) = 1] = \frac{1}{2}$ as:
   - if y, z are chosen randomly, no information can be extracted from $m \cdot g^z$, i.e. $m \cdot g^z$ is also a uniformly random element of Z_q and is independent from m.
2. If $h_3 = g^{xy}$, then $(c_1, c_2) = (g^y, m_b \cdot g^{xy})$, which means that it is the Π ElGamal encryption scheme.
   - $\Pr[D(g^x, g^y, g^{xy}) = 1] = \Pr[PubK^{cpa}_{A,\Pi}(n) = 1]$
   - We have to prove that this probability is close to $\frac{1}{2}$

Thus:

$$|\frac{1}{2} - \Pr[PubK^{cpa}_{A,\Pi} = 1]| = \tag{1}$$

$$|\Pr[D(g^x, g^y, g^z) = 1] - \Pr[D(g^x, g^y, g^{xy}) = 1] \leq \mathsf{negl}(n) \tag{2}$$

by the hardness assumption of DDH.

□

4 RSA

Rivest-Shamir-Adleman

Definition 4.1 (Textbook RSA)**.**

- Gen:
  - Input: 1^n
  - Output: (N, e, d), where e is the public key, and d is the secret key
  - Output is generated as follows:
    1. Choose two primes p, q of bitsize n
    2. $N = p \cdot q, \varphi(n)$(Euler's totient function) $= (p-1)(q-1)$
    3. Choose e such that $\gcd(e, \varphi(n)) = 1$

4. Compute $d \equiv e^{-1} \pmod{\varphi(N)}$, i.e. d is the solution of $e \cdot x \equiv 1 \pmod{\varphi(N)}$
5. $pk = (N, e), sk = d$

- Enc:
  - Input: $pk = (N, e), m \in \mathbb{Z}_N^*$ (where $\mathbb{Z}_n^* = \{1 \leq k \leq N \cdot \gcd(k, N) = 1\}$)
  - Output: $c = m^e \pmod{N}$
- *Dec*:
  - Input: $sk = d, c$
  - Output: $c^d \pmod{N}$

Correctness:

- $c^d \equiv (m^e)^d \equiv m^{ed} \pmod{N} = m^{1+k\varphi(N)}$ (by the choice of d) $= m \cdot (m^{\varphi(N)})^k \equiv m \pmod{N}$
- Where $m^{\varphi(N)} = 1$ by the Euler-Fermat theorem.

Recall 4.1 (Euler-Fermat theorem)**.** For any N and $\gcd(a, N) = 1$:

$$a^{\varphi(n)} \equiv 1 \pmod{N}$$

Remark 4.1.

- We need $\gcd(e, \varphi(N)) = 1$ to have a solution for d.
- We required that $\gcd(m, N) = 1$, but usually, we relax this requirement, since:

$$\Pr[\gcd(m, N) \neq 1](\text{either } p|m \text{ or } q|m) = \frac{q+p}{N} \approx \frac{1}{p} \approx 2^n = \mathsf{negl}(n)$$

Public Key Crypto notes - 04

By contributors

March 3. 2026

1 Security of RSA

The security of RSA relies on the computational hardness of factoring large composite integers (compute p, q from N).

1.1 The Factorization Problem

Definition 1.1 (Factorization Experiment)**.** Let $\mathsf{Factor}_A(n)$ be the following experiment:

1. Run $\mathsf{Gen}(1^n)$ to obtain an RSA modulus $N = pq$, where p, q are primes.
2. The adversary A is given N and outputs an integer p'.
3. If $p' \mid N$ and $1 < p' < N$, then $\mathsf{Factor}_A(n) = 1$; otherwise, $\mathsf{Factor}_A(n) = 0$.

Definition 1.2 (Hardness of Factorization)**.** The factorization problem is said to be *hard* if for all probabilistic polynomial-time (PPT) adversaries A,

$$\Pr[\mathsf{Factor}_A(n) = 1] \leq \mathsf{negl}(n).$$

Remark 1.1. In general, factorization is easy for random integers. For example, for a uniformly random integer N,

$$\Pr[N \text{ is even}] = \tfrac{1}{2},$$

in which case $p = 2$ is a trivial factor. However, when N is generated as a product of two large primes $N = pq$, factorization is believed to be computationally hard.

1.2 Algorithms for Factorization

1.2.1 Trial Division (Brute-force)

- Try all integers $p = 2, 3, \ldots, \sqrt{N}$
- If some p divides N, output p

Remark 1.2. The running time of trial division is

$$\mathcal{O}(\sqrt{N}) = \mathcal{O}(2^{n/2}),$$

which is exponential in the bit-length n of N.

1.2.2 Baby-Step Giant-Step Method for Factorization

Let p, q be two primes of bit-length approximately n, i.e.,

$$2^n \leq p, q < 2^{n+1},$$

and let

$$N = pq.$$

Recall that Euler's totient function satisfies

$$\varphi(N) = (p-1)(q-1) = pq - (p+q) + 1 = N - S + 1,$$

where

$$S = p + q.$$

Claim 1.1. Given N and $S = p + q$, one can compute p and q in polynomial time.

Proof. Consider the quadratic polynomial

$$f(x) = (x-p)(x-q).$$

Expanding, we obtain

$$f(x) = x^2 - (p+q)x + pq = x^2 - Sx + N.$$

Thus, finding the roots of $f(x)$ yields p and q. Since quadratic equations can be solved efficiently, p and q can be recovered in polynomial time given N and S. □

Remark 1.3. As S has size 2^{n+2}, we can write $m = [2^{\frac{n+2}{2}}]$ (the integer part of m) and write S in the form:

$$S = i \cdot m + j,$$

where $0 \leq i, j \leq m$. This idea underlies baby-step giant-step style approaches to factorization.

1.3 Baby-Step Giant-Step via Euler–Fermat

Recall Euler's theorem: for all a such that $\gcd(a, N) = 1$,

$$a^{\varphi(N)} \equiv 1 \pmod{N}.$$

Since $\varphi(N) = N - (p+q) + 1$, we obtain

$$a^{N-S+1} \equiv 1 \pmod{N},$$

where $S = p + q$.

1.3.1 Baby-Step Giant-Step Algorithm

- Choose $x \in_R \mathbb{Z}_N^*$ uniformly at random.
- Fix a parameter m.
- Compute the following values:

$$x_i = (x^m)^i \quad \text{for } 0 \leq i \leq m$$

$$x'_j = x^{N+1-j} \quad \text{for } 0 \leq j \leq m$$

We look for a collision:

$$x_i = x'_j \quad \text{for some } i, j.$$

1.3.2 Collision Illustration

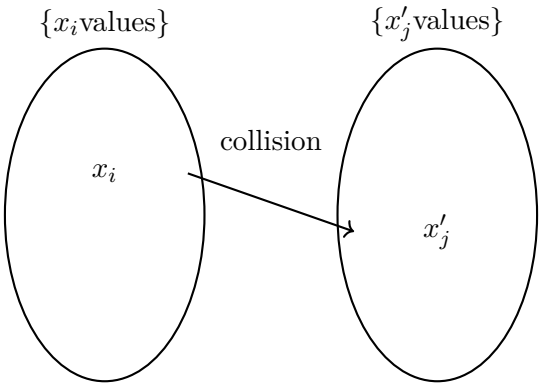


Figure 1: Baby-step giant-step collision: $x_i = x'_j$

1.3.3 Extracting $S = p + q$

A collision implies:

$$x^{i \cdot m} = x^i = x'_j = x^{N+1-j} \pmod{N}.$$

Rewriting,

$$x^{N+1-(i \cdot m+j)} \equiv 1 \pmod{N}.$$

Thus,

$$S = i \cdot m + j.$$

1.4 Remarks on the Method

Remark 1.4. This is only a sketch. It may happen that

$$x^a \equiv 1 \pmod{N}$$

for some $a < \varphi(N)$. In that case, we only recover $S \bmod a$. However, the probability of this happening is not too large, and one can retry with a different choice of x. Even in such cases, partial information about S may still be useful.

Remark 1.5. The time and space complexity of this approach is

$$\mathcal{O}(m) = \mathcal{O}(2^{n/2}) = \mathcal{O}(\sqrt[4]{N}).$$

Remark 1.6. There exist algorithms such as Pollard's ρ method with time complexity

$$\mathcal{O}(\sqrt[4]{N})$$

but only constant space complexity $\mathcal{O}(1)$.

1.5 Best Known Algorithms

Remark 1.7. The best known general-purpose factorization algorithm is the *General Number Field Sieve (GNFS)*, with running time

$$\mathcal{O}(e^{(\sqrt[3]{\frac{64}{9}}+\epsilon)(\ln N)^{\frac{1}{3}}(\ln\ln N)^{\frac{2}{3}}})$$

Remark 1.8 (Key Size Implications)**.** To achieve a security level comparable to AES-128, one needs an RSA modulus of size approximately

$$N \approx 5000 \text{ bits}.$$

1.6 Choice of the Public Exponent

Remark 1.9 (Choosing the Exponent e)**.** If e is very small, for example $e = 3$, and the message m is small, then interpreting m as an integer we may have

$$c = m^e < N.$$

In this case, m can be recovered directly by computing

$$m = \sqrt[e]{c}.$$

A standard choice for the public exponent is

$$e = 2^{16} + 1 = 65537.$$

This value is prime, and computing

$$c = m^e \bmod N$$

is efficient, since

$$m^{2^{16}+1} = m \cdot m^{2^{16}} = m \cdot ((m^2)^2 \cdots),$$

which can be computed using repeated squaring (about 16 squarings).

1.7 CPA Security of RSA

Recall the CPA (chosen-plaintext attack) experiment:

1. A key pair (pk, sk) is generated.
2. The adversary A is given pk and outputs two messages m_0, m_1.
3. A random bit $b \in \{0, 1\}$ is chosen and
$$c = \mathsf{Enc}(pk, m_b)$$
is computed.
4. The adversary A is given c and outputs a guess b' for b.

Remark 1.10. Textbook RSA is *not* CPA-secure.

Indeed, in step 3,

$$c_0 = m_0^e \bmod N, \qquad c_1 = m_1^e \bmod N,$$

and the adversary can simply output b' such that

$$c = c_b.$$

Remark 1.11. By randomizing the encryption, one can obtain CPA-secure encryption schemes.

1.8 Hard-Core Bits of RSA

Theorem 1.1. Computing x from $(N, e, x^e \bmod N)$ is as hard as computing the least significant bit $\mathrm{lsb}(x)$ of x.

That is, if there exists a PPT algorithm A such that

$$A(N, e, x^e \bmod N) = \mathrm{lsb}(x),$$

then there exists a PPT algorithm B such that

$$B(N, e, x^e \bmod N) = x.$$

Remark 1.12. The bit $\mathrm{lsb}(x)$ is called a *hard-core bit* of RSA.

1.9 Making RSA CPA-Secure

Remark 1.13. One can construct a CPA-secure encryption scheme from RSA by encrypting a randomized message. Let

$$m' = r \,\|\, m,$$

where r is chosen uniformly at random. The ciphertext is then computed as

$$c = (m')^e \bmod N.$$

Public Key Crypto notes - 05

By contributors

March 09. 2026

1 Various theorems

Theorem 1.1. Computing x from N, e and $x^e \pmod N$ is as hard as computing $lsb(x)$.

Proof. Let A be an adversary such that $A(r^e \pmod N) = lsb(r)$ for any $r \in \mathbb{Z}_N^*$. We use A to recover x from $x^e \pmod N$:

1. $lsb(x)$ is given by $A(x^e \pmod N)$.
2. To compute $2nd(x)$ (which is the second least significant bit), let $y = x^e \pmod N$.
3. Cases:
   (a) $lsb(0) \to x$ is even $\to 2nd(x) = lsb(\frac{x}{2})$. Note that $A(\frac{y}{2^e}) = A((\frac{x}{2})^e) = 2nd(x)$.
   (b) $lsb(x) = 1 \to x$ is odd. $\frac{x}{2} \pmod N = \frac{x+N}{2}$ (since we are working in modular arithmetic). We are trying to find:

$$lsb(\frac{x}{2}) \pmod N = 2nd(x+N) = \tag{1}$$

$$= 2nd(x) + 2nd(N) + 1 \pmod 2 \tag{2}$$

So:

$$2nd(x) = lsb(\frac{x}{2} \pmod N) + 2nd(N) + 1 \pmod 2 = \tag{3}$$

$$= A(\frac{y}{2^e} \pmod N) + 2nd(N) + 1 \pmod 2 \tag{4}$$

□

Remark 1.1. If $m \in \{0,1\}$ and $c = (r||m)^e \pmod N$ where r is a random integer of bit size $2n-1$, then it is a CPA-secure encryption.

2 Padding schemes

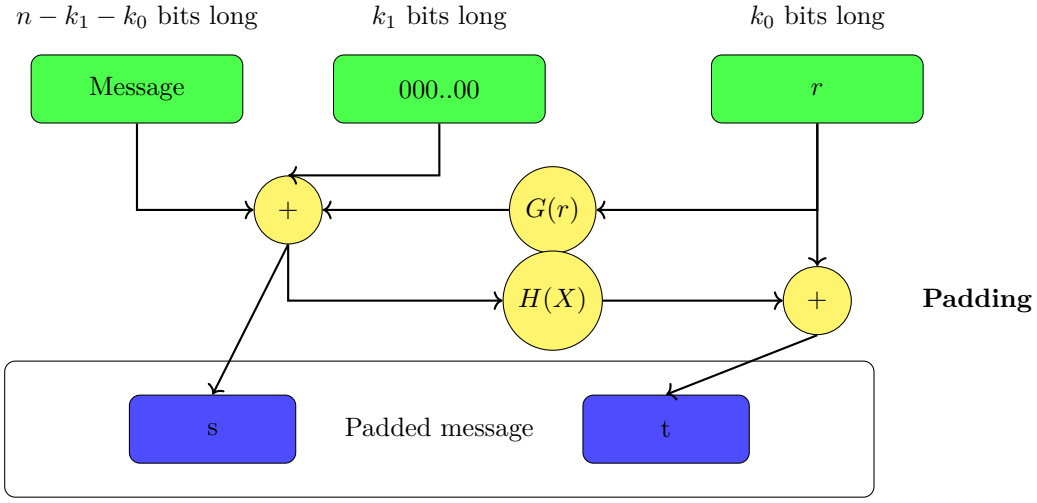
2.1 RSA PKCS#1 v1.5: Padded RSA

$\hat{m} = (0, \_, 0||r||0, \_, 0||m)$ with random r ($\hat{m}$ is the padded message).
This scheme has some security disadvantages[1].

[1] https://medium.com/@c0D3M/bleichenbacher-attack-explained-bc630f88ff25

2.2 RSA-OAEP-optimal asymmetric encryption padding

Let G, H be two secure hash functions.



That is:

$$s = m \oplus G(r) \tag{5}$$
$$t = r \oplus H(s) \tag{6}$$

and $\hat{m} = s||t$, and encrypt $\hat{m}$ with RSA.

3 Summary of learnt public key encryption schemes so far

Property	**ElGamal**	**RSA**
Security parameter	n	n
Public keys	p, g, g^x	N, e
Public key size	$3n$	$2n + c$
Secret key	x	d
Secret key size	n	n
Ciphertext	$(g^y, g^{xy} \cdot m)$	$m^e \bmod N$
Ciphertext size	$2n$	n

Table 1: Comparison of ElGamal and RSA public key encryption schemes

4 Digital signatures

Definition 4.1 (Digital signature scheme)**.** A digital signature scheme Π consists of three PPT algorithms: $\Pi = (\mathsf{Gen}, \mathsf{Sig}, \mathsf{Ver})$.

- Gen:
  - Input: 1^n
  - Output: pk, sk

- Sig:
  - Input: m, sk
  - Output: $\sigma \leftarrow \mathsf{Sig}_{sk}(m)$
- Ver:
  - Input: σ, pk, m
  - Output: $b = \begin{cases} 1, \text{if valid} \\ 0, \text{if invalid} \end{cases}$

We require, except with negligible probability, that $\mathsf{Ver}_{pk}(m, \mathsf{Sig}_{sk}(m)) = 1$.

Definition 4.2 (Security of a digital signature scheme)**.**

- Adversary A knows the public parameters Π, pk.
- A knows many (m, σ) pairs.

Our goal is to prevent a PPT adversary A to obtain the following:

- A signature of a *new* message m.
- A *new* signature of an *old* message.

We do *not* care about replay attacks.

Definition 4.3 (The signature forging experiment)**.** The signature forging experiment Sig-forge$_{A,\Pi}(n)$ is the following:

1. Run $G(1^n)$ to obtain (pk, sk)
2. A is given pk, and oracle access to $\mathsf{Sig}_{sk}(.)$
3. Let Q be the set of all queries that A submitted to the oracle and also the oracle's answers.
4. A outputs (m, σ).
5. Sig-forge$_{A,\Pi}(n) = \begin{cases} 1, if \mathsf{Ver}(m, \sigma) = 1 \cap (m, \sigma) \notin Q \\ 0 \text{ otherwise} \end{cases}$

Definition 4.4 (Security of a digital signature scheme)**.** A digital signature scheme $\Pi(\mathsf{Gen}, \mathsf{Sig}, \mathsf{Ver})$ is secure if for all PPT adversary A:

$$\Pr[\text{Sig-forge}_{A,\Pi}(n) = 1] \leq \mathsf{negl}(n)$$

4.1 The RSA-signature scheme

Definition 4.5 (Plain RSA-signature scheme)**.**

Gen:

- Input: 1^n
- Output: $pk = (N, e), sk = d$ as in RSA.

Sig:

- Input: $m \in \mathbb{Z}_n^*, sk = d$
- Output: $m, \sigma = m^d \pmod N$

Ver:

- Input: $(m, \sigma), pk = (N, e)$
- Output: Verifies, whether $m = \sigma^e \pmod N$

Remark 4.1. For a given m, it is as hard to obtain σ and breaking RSA. However, one can easily generate valid (m, σ) pairs: If A knows at least two documents and corresponding signatures $(m_1, \sigma_1), (m_2, \sigma_2)$, then $(m, \sigma) = (m_1 m_2, \sigma_1 \sigma_2)$ is also valid: $m^d = (m_1 m_2)^d = m_1^d m_2^d = \sigma_1 \sigma_2$.

Definition 4.6 (RSA-FDH (Full Domain Hash) signature)**.**

- Let $H : \{0,1\}^* \to \mathbb{Z}_N^*$ be a hash function.
- Gen:
  - Input: 1^n
  - Output: $pk = (N, e), sk = d$
- Sig:
  - Input: m, sk
  - Output: $m, \sigma = H(m)^d$
- Ver:
  - Input: m, σ, pk
  - Output: $H(m) = \sigma^e \pmod N$

Remark 4.2. One can prove the security of this scheme. It is also called the hash-and-sign paradigm.

Public Key Crypto notes - 06

By contributors

March 16. 2026

1 Proof of knowledge protocols

- Goal: Design a signature scheme that utilises the discrete log problem.
- Idea: Digital signature: Proof of knowledge of secret key without telling any information about it.
- Example: Ali Baba's cave: Bob can only meet Alice from A when entering from B if he knows the password. Alice does not hear the password.
- The probability of passing is 50%. If Bob passes the test two times, 25%, etc.
- Given n tests, Bob can pass the test by randomly guessing with $\frac{1}{2^n}$ probability.
- This is a proof to the validator, Alice. For outside observers, no one can tell whether Bob really knows the password.
- Three steps:
  1. Commitment: When the prover makes an action (Bob enters the cave).
  2. Challenge: When the validator asks the direction from Bob (*"Enter from A and go through B"*).
  3. Response: When Bob comes from the corresponding direction.

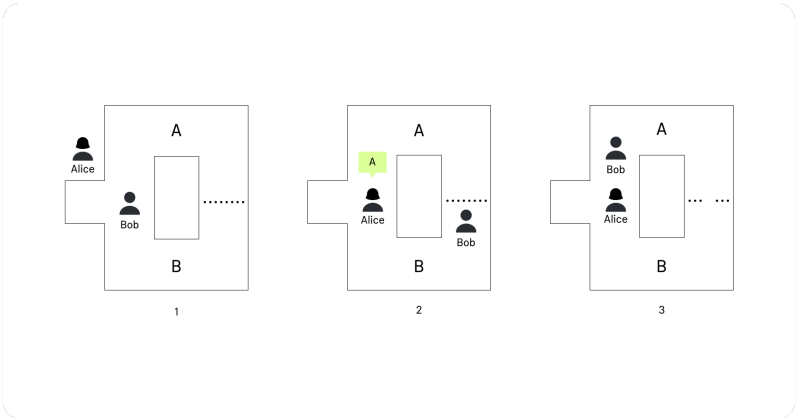


Figure 1: Ali Baba cave example

1.1 Schnorr identification scheme

Let $pk = (p, g, q, h = g^x), sk = x$.

- Goal: Prove the knowledge of sk under pk without revealing any information on sk.
- Protocol:
  1. Prover chooses $k \in_R \mathbb{Z}_q$ and $I = g^k$ and sends I to the verifier. (Commitment)
  2. Verifier chooses $r \in_R \mathbb{Z}_q$ and sends this to the Prover. (Challenge)
  3. Prover computes $s = rx + k$ and sends s to the Verifier. (Response)
  4. Verifier checks: $I = g^s \cdot h^{-r}$. If yes, the proof is accepted.
- Correctness: $g^s \cdot h^{-r} = g^{rx+k} \cdot (g^x)^{-r} = g^{rx+k-rx} = g^k = I$

Theorem 1.1. If the Prover can give a valid response for any challenge, then the Prover knows x.

Proof. Let $I = g^k$ and let r_1, r_2 be two *different* challenges with respect to I and let s_1, s_2 be the two responses.

$$s_1 = r_1 x + k \tag{1}$$
$$s_2 = r_2 x + k \tag{2}$$
$$s_1 - s_2 = (r_1 - r_2)x \tag{3}$$

So:

$$x = \frac{s_1 - s_2}{r_1 - r_2}$$

□

Remark 1.1. If the Prover knows r in advance (before the commitment), they can generate a valid response s without knowing secret x.
Indeed: For a given challenge r, let $s \in_R \mathbb{Z}_q$ and $I = g^s \cdot g^{-r}$.

Note that this is an interactive protocol, since the Verifier challenges the Prover. The Verifier's role is to generate the challenge. How can we make it non-interactive?

1.2 Fiat-Shamir transform

Replace the challenge r with $r = H(I)$, where H is a secure cryptographic hash function.
To prove security, we use the Random Oracle Model (ROM), where $H(.)$ is replaced by a random oracle.

Definition 1.1 (Random Oracle Model (ROM))**.** For a given input x, if it has not yet been queried, then let $r \in_R \mathbb{Z}_q$, and put $r = \mathcal{O}(x)$, where $\mathcal{O}$ is the random oracle. If it has been queried, return the same output.

1.3 Schnorr signature scheme

- Gen:
  - Input: 1^n
  - Output: $pk = (p, g, q, h = g^x), sk = x$
- Sig:
  - Input: sk, m
  - Output: (I, s), where:

$$k \in_R \mathbb{Z}_q, \tag{4}$$
$$I = g^k, \tag{5}$$
$$r = H(I, m), \tag{6}$$
$$s = rx + k \tag{7}$$

- Ver:
  - Input: $pk, m, (I, s)$
  - Output: Compute $r = H(I, m)$ and check whether $I = g^s \cdot h^{-r}$

Theorem 1.2. The Schnorr signature scheme is secure, provided that $H(.)$ is modelled as a random oracle and the discrete logarithm problem is computationally hard.

Proof. The proof uses the rewind technique. We prove by reduction. Assume we have a PPT adversary A who can generate valid signatures with non-negligible probability. We use this A to compute the discrete logarithm.

For simplicity, we assume that A is *perfect*, meaning that it *always* returns a valid signature. Reduction: Given $pk = (p, g, q, h = g^x)$. Our goal is to learn x. TODO: Ábra a füzetben

1. Give pk to A.
2. Give A oracle access to $\mathcal{O}$, and let Q be the set of queries submitted by A.
3. A generates I and m with $(I, m) \notin R$ and queries the output $r = \mathcal{O}(I, m)$.
4. A returns (I, s).

Now, rewind it to step 3. and get (i, s', r') at step 4. We have valid (r, s) and (r', s'), with the same I, i.e. $s = rx + k$ and $s' = r'x + k$ with some k, such that $I = g^k$.

This means that we can compute x. □

Remark 1.2. This signature algorithm has been patented (expired in 2010). Instead, NIST designed a new and free signature algorithm called Digital Signature Algorithm (DSA).

Public Key Crypto notes - 07

By contributors

March 23. 2026

1 Schnorr-signature recap

- Gen:
  - Input: 1^n
  - Output: $pk = (p, g, q, h = g^x), sk = x$
- Sig:
  - Input: sk, m
  - Output: (I, s), where:

$$k \in_R \mathbb{Z}_q, \tag{1}$$
$$I = g^k, \tag{2}$$
$$r = H(I, m), \tag{3}$$
$$s = rx + k \tag{4}$$

- Ver:
  - Input: $pk, m, (I, s)$
  - Output: Compute $r = H(I||m)$ and check whether $I = g^s \cdot h^{-r}$

This scheme is patented. Instead of this scheme, we mostly use DSA (made by NIST).

2 Digital Signature Algorithm (DSA)

- Designed by NIST.
- Let $H : \{0,1\}^* \to \mathbb{Z}_q, F : \{0,1\}^* \to \mathbb{Z}_q$ be two functions. Consider the following three algorithms:
- Gen:
  - Input: 1^n
  - Output: Public key $pk = (p, g, q, h = g^x)$, Secret key: $sk = x$
- Sig:
  - Input: m, sk

  - Output: (r, s)
  - $k \in_R \mathbb{Z}_q, r = F(g^k), s = k^{-1}(H(m) + xr) \pmod q$

- Ver:
  - Input: $m, (r, s), pk$
  - Output: $r \stackrel{?}{=} F(g^{H(m)\cdot s^{-1}} \cdot h^{r\cdot s^{-1}})$

- Correctness:

$$g^{H(m)\cdot s^{-1}} \cdot h^{r\cdot s^{-1}} = \tag{5}$$
$$g^{H(m)\cdot s^{-1}+x\cdot r\cdot s^{-1}} = \tag{6}$$
$$g^{s^{-1}(H(m)+x\cdot h)} = \tag{7}$$
$$g^{\frac{k}{H(m)+xr}\cdot(H(m)+xr)} = g^k \tag{8}$$
$$F(g^k)\checkmark \tag{9}$$

Theorem 2.1 (Security of the DSA scheme)**.** If H and F are modelled by a random oracle and the discrete logarithm problem is hard, then DSA is secure.

Remark 2.1. In practice, $F(x) = x \pmod q$, this means that the "random oracle" is not really random.

Remark 2.2. If the same k is used multiple times, then one can obtain $sk = x$. It is important to use a different k for each iteration.
Indeed:

$$s_1 = k^{-1}(H(m_1) + x + r) \tag{10}$$
$$s_2 = k^{-1}(H(m_2) + x + r) \tag{11}$$
$$s_1 - s_2 = k^{-1}(H(m_1) - H(m_2)) \to k \text{ can be computed} \tag{12}$$
$$\to x \text{ can also be computed.} \tag{13}$$

Note: Playstation 3: All k used for updates were the same.[1]

2.1 Parameter selection

- p is large, $\approx 3000 - 4000$ bits,
- q is medium size (size ≈ 256 bits),
- $\to$ signature size: 2×256 bits ≈ 500 bits.

[1] https://deeprnd.medium.com/decoding-the-playstation-3-hack-unraveling-the-ecdsa-random-generator-flaw-e9074a51

3 Further digital signatures using the discrete log problem

General form of signatures:

- Given $a, b, c : ak = b + cx \pmod{q}$
- Verification: $g^{ak} \stackrel{?}{=} g^b \cdot g^{xc}$
- Examples:
  1. Schnorr:
     - $a = -1$
     - $b = s$
     - $c = -r$
  2. DSA:
     - $a = s$
     - $b = H(m)$
     - $c = (g^k \pmod{p}) \pmod{q}$
  3. ElGamal:
     - $a = s$
     - $b = h(m)$
     - $c = -r \pmod{p-1}$

4 Exotic protocols

4.1 Blind signature

- Given: signer S and user U.
- U has a message m and wants S to sign it without telling m to S.

4.2 Blind RSA signature

- The signer has plain RSA keys, where $pk = (N, e), sk = d$.
- U has message $m \in \mathbb{Z}_N^*$, blinding parameter $r \in_R \mathbb{Z}_N^*$, and sends $m' = m \cdot r^e \pmod{N}$ to S.
- S signs m', and sends $s' = m'^d \pmod{N}$ to U.
- So $s = s' \cdot r^{-1} \pmod{N} = m^d (r^e)^d r^1 = m^d$

Remark 4.1. The value r is the blinding parameter. If it is chosen randomly, then m' cannot leak any information about m.

4.3 Blind Schnorr signature

- S chooses $k \in_R \mathbb{Z}_q$, sends $I = g^k$ to U.
- U chooses blinding parameters $\alpha, \beta \in_R \mathbb{Z}_q$.
- U calculates:

$$I' = I \cdot g^\alpha \cdot h^\beta \tag{14}$$
$$r' = H(I'||m) \tag{15}$$
$$r = r' + \beta \tag{16}$$

  and sends r to S.
- S calculates $s = rx + k$ and sends it to U.
- U calculates $s' = s + \alpha$
- The signature is $sig = (I', s')$

Correctness:

$$g^s \stackrel{?}{=} I \cdot h^r = \tag{17}$$
$$g^{s'-\alpha} \stackrel{?}{=} I' \cdot g^{-\alpha} \cdot h^{-\beta} \cdot g^{r'+\beta} = \tag{18}$$
$$g^{s'} \stackrel{?}{=} I' \cdot h^{r'} \tag{19}$$

4.4 Oblivious transfer

There are two parties: sender A and receiver B. A wants to send a message (out of two) to B, such that:

- B can only receive one of the messages.
- A should *not* learn which message B received.
- Formally:
  - Two messages: m_0, m_1
  - B chooses $b \in_R \{0, 1\}$ and receives m_b

4.4.1 Protocol 1 (Even, Goldreich, Lempel)

- Based on RSA.
- A has two messages: m_0, m_1 and generates RSA keys $pk = (N, e), sk = d$.
- A chooses $x_0, x_1 \in_R \mathbb{Z}_N^*$ and sends them to B along with pk.
- B chooses $b \in_R \{0, 1\}$ and $k \in_R \mathbb{Z}_N^*$, and computes $v = (x_b + k^e) \pmod N$ and sends v (also called a *pre-envelope*) to A.

- A computes:

$$k_0 = (v - x_0)^d, \tag{20}$$
$$k_1 = (v - x_1)^d, \tag{21}$$
$$m_0' = m_0 + k_0, \tag{22}$$
$$m_1' = m_1 + k_1, \tag{23}$$
$$\tag{24}$$

  and sends m_0', m_1' to B.

- B computes $m_b = m_b' - k$

But B cannot compute $k_{1-b} = (v - x_{1-b})^d$

Remark 4.2. This is a one-out-of-two oblivious transfer, but there are 1-out-of-n or m-out-of-n oblivious transfer schemes out there also.

Public Key Crypto notes - 08

By contributors

March 30. 2026

1 Oblivious transfer recap

A sends m_0, m_1, B chooses $b \in_R \{0,1\}$ and obtains m_b. RSA-based protocol.

2 Protocol 2 – Bellari-Micali

- Diffie-Hellman-based, interactive protocol.
- Let p, g, q be as in ElGamal.
- A chooses $c \in_R < g >$, where $< g >= \{g^i : i = 0, \ldots, q-1\}$ and sends c to B.
- B chooses $k \in_R \mathbb{Z}_q, b \in_R \{0,1\}, y_b = g^k, y_{1-b} = \frac{c}{g^k}$, and sends y_0, y_1 to A.
- A checks $y_0 \cdot y_1 \stackrel{?}{=} c$ and chooses two random exponents $r_0, r_1 \in_R \mathbb{Z}_q$
  - $c_0 = (g^{r_0}, H(y_0^{r_0}) \oplus m_0)$
  - $c_1 = (g^{r_1}, H(y_1^{r_1}) \oplus m_1)$
  - A sends c_0, c_1 to B
- B calculates $c_b = (v_0, v_1), m_b = H(v_0^k) \oplus v_1$

3 Application of oblivious transfers

3.1 Secure multiparty computations (MPC)

- There are two parties A, B. Both of them have a secret input x_A, x_B. There is a function $f(x, x')$. A and B want to evaluate $f(x_A, x_B)$ without them learning the other's secret.
- Example: Dating protocol
  - Two parties: A, B
  - Let $d_A = \begin{cases} 1 \text{ if } A \text{ wants to date with B} \\ 0 \text{ otherwise} \end{cases}$ and, $d_B = \begin{cases} 1 \text{ if } B \text{ wants to date with A} \\ 0 \text{ otherwise} \end{cases}$
  - The function to evaluate this: $\text{AND}(d_A, d_B)$
- Evaluating AND with oblivious transfer:
  - If $d_A = 1$, let $m_0 = 0$ (meaning that B chose 0), $m_1 = 1$
  - If $d_A = 0$, let $m_0 = 0, m_1 = 0$
  - Then, A sends m_0, m_1 to B by oblivious transfer

3.2 Group signatures

- Group/organization
- A Group Manager can add users to a group
- The group $(S_0, S_1, \dots S_n)$ jointly issues signature (m, σ) on behalf of the group itself
- The exact individual who actually made the signature is masked
- Requirements:
  - σ cannot be linked to any S_i
  - The Group Manager M can resolve σ (i.e. can learn the actual signer S_i)
- Algorithms:
  - Setup: M sets the parameters up
  - Joining member S_i is added by M (can even happen dynamically)
  - Signing: S_i signs m and outputs σ
  - Verification: Third-party user U verifies by $\mathsf{Ver}(m, \sigma)$
  - Open: M links the signature σ to S_i

3.2.1 Example of group signature protocols

Chaum's group signature, '91:

- Setup/Join:
  - S_i generates ElGamal keys: $pk_i = (p, g, q, g^{x_i}), sk_i = x_i$
  - S_i gives pk_i to M
- Sign:
  - At period t, M chooses $r_{t,i} \in_R \mathbb{Z}_q$, and sends it to S_i
  - M publishes $P_t = \{g^{r_{t,i} \cdot x_i}, i = 1, 2, \dots\}$ (all blinded public keys at period t)
  - S_i uses $r_{t,i} \cdot x_i$ as a secret key for the ElGamal signature, $pk = g^{r_{t,i} \cdot x_i}$.
- Verify:
  - U checks: $pk \stackrel{?}{\in} P_t$ and verifies the ElGamal signature.
- Open:
  - M checks for which $i : pk = (g^{x_i})^{r_{t,i}}$

Remark 3.1. At period t, only *one* signer can make a signature. Hence, the whole signature (m, σ, P_t) is linear in the size of the group.

Before protocol 2, we need preparation with proof of knowledge protocols.

3.3 Proof of knowledge protocols

3.3.1 Schnorr's PoK protocol

- Schnorr PoK$[x : g^x = h](m)$
- Prover chooses $k \in_R \mathbb{Z}_q$, sends $I = g^k$ to the Verifier.
- Verifier chooses $r \in_R \mathbb{Z}_q$ or $r = H(I)$ or $r = H(I||m)$, and sends r to the Prover
- Prover calculates $s = rx + k$, and sends s to the Verifier
- The Verifier checks whether $r \stackrel{?}{=} H(g^s \cdot h^{-r}||m)$, where $g^s \cdot h^{-r} = I$

Remark 3.2. Success probability without knowing x is $\approx \frac{1}{q}$.

3.3.2 PoK of LogLog

Goal: Prover knows x such that $y = g^{(a^x)}$ with given g, y, a.

- Prover chooses $k \in_R \{0, \dots, 2^\lambda\}, t = g^{a^k}$ and t is sent to the Verifier
- The Verifier chooses $r \in_R \{0, 1\}$ and sends it to the Prover
- The Prover calculates: $s = \begin{cases} k, \text{ if } r = 0 \\ k - x, \text{ if } r = 1 \end{cases}$ and sends s to the Verifier
- Verifier performs the following: $t = \begin{cases} g^{(a^s)}, \text{ if } r = 0 \\ y^{(a^s)}, \text{ if } r = 1 \end{cases}$
- Correctness:
  - $r = 0$ ✓
  - $r = 1 \rightarrow y^{(a^s)} = y^{(a^{k-x})} = [g^{(a^x)}]^{(a^{k-x})} = g^{(a^x)\cdot(a^k)\cdot(a^{-x})} = g^{(a^k)} = t$

3.3.3 Remarks

Remark 3.3. If $r = 0$, the Verifier checks if t has the right form. If $r = 1$, then the Verifier checks if P knows x, provided t has the right form.

Remark 3.4. Success probability is $\frac{1}{2}$.

3.3.4 Non-interactive PoK protocol

PoK$[x : y = g^{(a^x)}](m = r, s_1, \dots, s_l) \in \{0, B^l \cdot \mathbb{Z}_q^l\}$ such that $r = H(m||y||g||a||t_1|| \dots ||t_l)$ with $t_i = \begin{cases} g^{(a^{s_i})}, \text{ if } r_i = 0 \\ y^{(a^{s_i})}, \text{ if } r_i = 1. \end{cases}$
Construction:

- $t_i = g^{(a^{k_i})}$ with random k_i and $s_i = \begin{cases} k_i, \text{ if } r_i = 0 \\ k_{i-x}, \text{ if } r_i = 1 \end{cases}$.

Public Key Crypto notes - 09

By contributors

April 13. 2026

1 PoK protocols we learnt about

Recall 1.1 (Proof-of-knowledge protocols)**.**

1. PoK$[x : g^x = h](m) \leftarrow$ Schnorr signature
2. PoK$[x : g^{a^x} = y](m)$, g, a, y are public

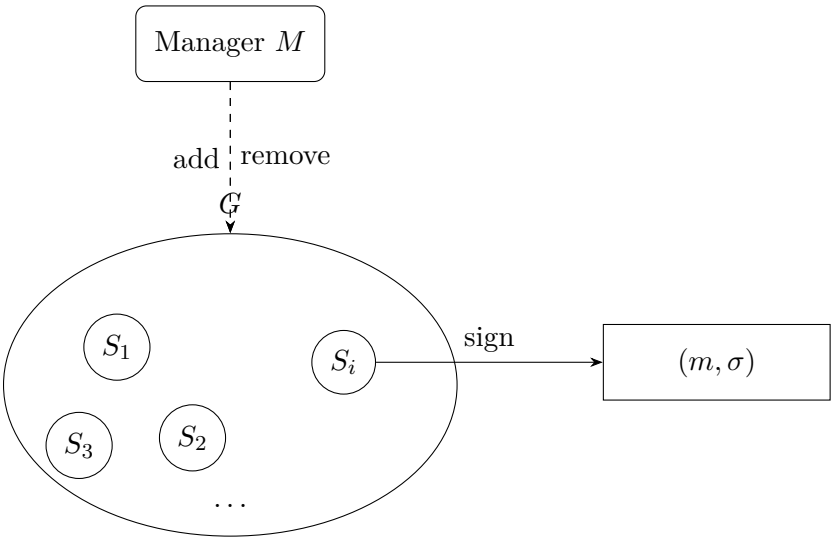
2 New PoK protocol - PoK RootLog

- Public parameters: g, e, y
- Goal: Prover knows x such that: $y = g^{(x^e)}$, where $ord(g) = N$ is an RSA modulus.
- Two parties: $Prover(x)$ and $Verifier(g, N, e, y)$ who know all public parameters.
  1. The Prover chooses $k \in_R 1, \dots, 2^\lambda$, $t = g^{(k^e)}$, where λ is a security parameter. The commitment t is sent to the Verifier.
  2. The Verifier chooses $r \in_R \{0, 1\}$ and sends it to the Prover.
  3. The Prover checks: $s = \begin{cases} k, & \text{if } r = 0 \\ \frac{k}{x}, & \text{if } r = 1 \end{cases}$, where x is the to-be-proved value of the Prover, and sends s to the Verifier.
  4. The Verifier checks: $t \stackrel{?}{=} \begin{cases} g^{(s^e)}, & \text{if } r = 0 \\ y^{(s^e)}, & \text{if } r = 1 \end{cases}$
- Correctness: $r = 0$✓
  $r = 1 : y^{(s^e)} = y^{(\frac{k}{x}^e)} = g^{(x^e)\cdot(\frac{k}{x})^e} = g^{(k^e)} = t$
- Success probability: $\frac{1}{2}$, so to achieve negligible probability that the Prover succeeds with proving an incorrect value, we need to iterate the protocol several times.

2.1 Non-interactive version

- For $i = 1, \dots$: $t_i = g^{(k_i)^e}$ with random k_i, $r = H(m||y||g||e||t_1||t_2|| \dots)$
- $s_i = \begin{cases} k_i, & \text{if } r_i = 0 \\ \frac{k_i}{x}, & \text{if } r_i = 1 \end{cases}$, where $r = r_1||r_2||r_3|| \dots$.
- PoK $x : y = g^{(x^e)}(m) = (r, s_1, s_2, \dots)$

3 Group signatures continued



- Setup:
  - M computes:
    - $(N, c), (N, d)$ RSA keys
    - A prime p and $g \in \mathbb{Z}_p^*$ such that $ord_p(g) = N$
    - $a \in \mathbb{Z}_N^*$ with large order
    - Public key: (p, g, N, e, a), secret key: d
- Join:
  - x_i random
  - $y_i = a^{x_i} \pmod N$
  - $z_i = g^{y_i}$, where z_i is called the membership key
  - (y_i, z_i) is sent to M with PoK$[\alpha : a^\alpha = y_i]$
  - M returns $v_i = (y_i + 1)^d$
  - M stores y_i in $\mathcal{Y}$
- Sign:
  - S_i computes:
    - $r \in_R \mathbb{Z}_N$
    - $\tilde{g} = g^r$ (blinded g)
    - $\tilde{z} = \tilde{g}^{y_i}$
    - $V_1 = \text{PoK}[\alpha : \tilde{z} = \tilde{g}^{a^\alpha}](m)$
    - $V_2 = \text{PoK}[\beta \cdot \tilde{z} \cdot \tilde{g} = \tilde{g}^{\beta^e}](m)$
    - Signature $= \tilde{g}, \tilde{z}, V_1, V_2$
- Open:
  - Given $(\tilde{g}, \tilde{z}, V_1, V_2)$

- Let $\mathcal{Y}$ be the list of y_i values
- Then, check: $\tilde{g}^{y_i} \stackrel{?}{=} \tilde{z}$ for all $y_i \in \mathcal{Y}$

- Correctness:
  - By V_1, $\tilde{g} \cdot \tilde{z}$ has the form $\tilde{g}^{y_i+1}$
  - By V_2, S_i knows $(y_i + 1)^d = V_i$

4 Elliptic Curve Cryptography (ECC)

4.1 Motivation

- Diffie-Hellman key exchange
- Two users, A, B, they have secret keys a, b, and public keys $g^a, g^b \pmod p$
- It is secure if the discrete log problem is hard
- Having index calculus p must have bit size 3000-4000 (almost one megabyte)
- Using elliptic curves, we can have sufficiently secure key sizes of 200-500 bits

Definition 4.1 (Elliptic curve)**.** An elliptic curve E over $\mathbb{R}$ is a set of points:

$$E = E_{A,B} = \{(x, y) : y^2 = x^3 + Ax + B\} \cup \{\infty\}$$

with $A, B \in \mathbb{R}$, such that $4A^3 + 27B^2 \neq 0$.

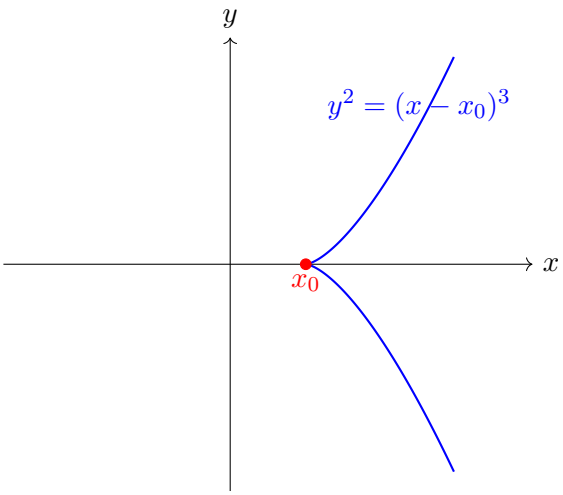


Figure 1: Singular cubic with a triple root translated to $x = x_0$.

Case 2: Normal elliptic curve

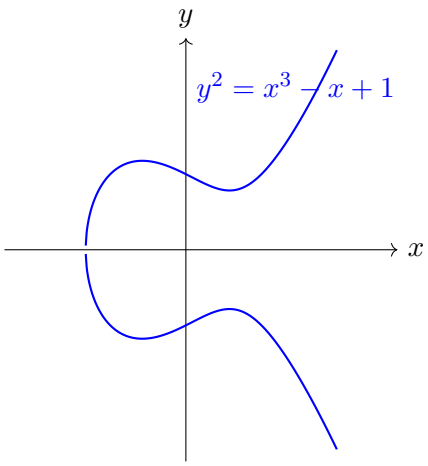


Figure 2: A normal (nonsingular) elliptic curve over $\mathbb{R}$.

Remark 4.1. E is symmetric to the x-axis: $(x, y) \in E$ if and only if $(x, -y) \in E$.

$f(x) = x^3 + Ax + B$ is cubic
Case 1: 3 different zeros
Case 2: 1 zero

- $D(A, B) = 4A^3 + 27B^2$ is the discriminant of the cubic $f(x) = x^3 + Ax + B$
- $f(x)$ has multiple zeros if $D(A, B) = 0$
- Case 1: $D(A, B) = 0, f(x) = (x - x_0)^3$
- Case 2: $D(A, B) = 0, f(x) = (x - x_0)^2(x - x_1)$

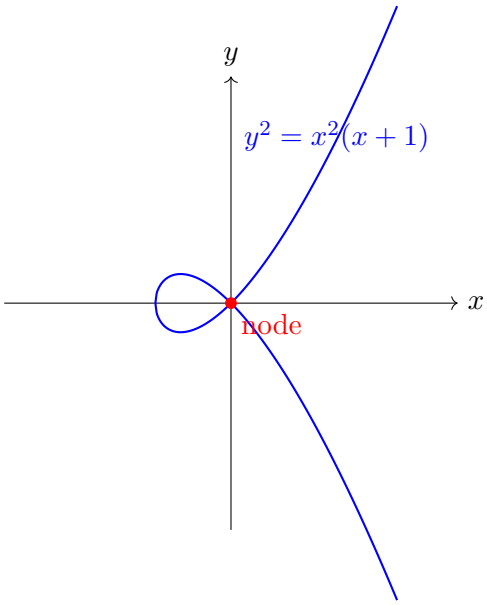


Figure 3: Singular cubic with a double root: a node.

4.2 Addition law of elliptic curves

- Let $P, Q \in E$, we define $P \oplus Q \in E$ (where $\oplus$ is the coordinate-wise addition) by the tangent-and-core method
- If $P = Q \rightarrow P \oplus Q = 2P$
- Point at infinity: basically the intersection of all vertical lines in the space. It is used when two points have the same y coordinate. The line between these two points reaches the point at infinity. In this case: $P \oplus Q = \infty, P \oplus \infty = P$

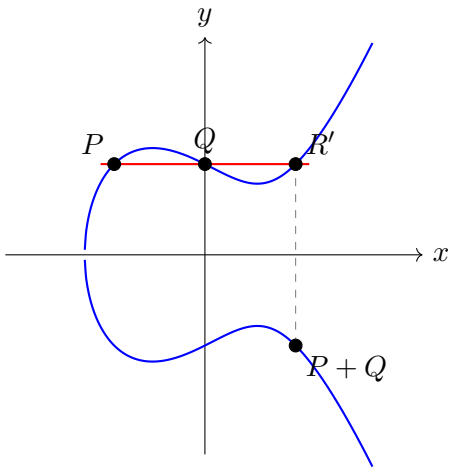


Figure 4: Geometric addition on the elliptic curve $y^2 = x^3 - x + 1$: the line through P and Q meets the curve again at R', and $P + Q$ is the reflection of R' across the x-axis.

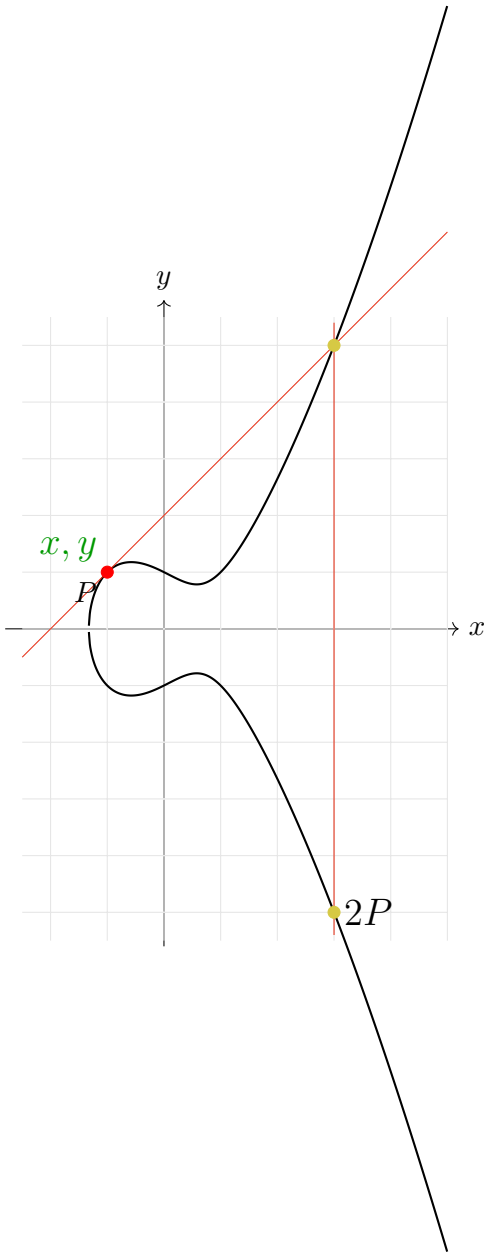


Figure 5: Point doubling on the elliptic curve $y^2 = x^3 - x + 1$: the tangent at $P = (-1, 1)$ meets the curve again at $R = (3, 5)$, and $2P = (3, -5)$ is the reflection of R across the x-axis.

Public Key Crypto notes - 10

By contributors

April 20. 2026

1 Elliptic curves

Recall 1.1. $E_{A,B} = \{(x,y) \in \mathbb{R}^2, y^2 = x^3 + Ax + B\} \cup \{\infty\}, 4A^3 + 27B^2 \neq 0$

For given $P = (x_P, y_P)$, $Q = (x_Q, y_Q)$, such that $P \neq Q, x_P \neq x_Q$. Let's compute $R = (x_R, y_R), R = P \oplus Q$.

Here, $-R = (x_R, -y_R)$, and $P, Q, -R$ are on E, and on the same line.

First, take the line through P, Q, $y = m(x - x_P) + y_P$ with slope m. Here, the slope $m = \frac{y_Q - y_P}{x_Q - x_P}$. Take the intersection of E with the line.

$$(m(x - x_P) + y_P)^2 = x^3 + Ax + B \tag{1}$$
$$x^3 + m^2x^2 + \ldots = 0 \tag{2}$$
$$(x - x_P)(x - x_Q)(x - x_R) = 0 \tag{3}$$
$$x^3 - (x_P + x_Q + x_R)x^2 + \ldots = 0 \tag{4}$$

So (2) and (3) are the same equation and $m^2 = x_P + x_Q + x_R$, i.e. $x_R = m^2 - x_P - x_Q$.

$-R = (x_R - y_R)$ is on the line. So $-y_R = m(x_R - x_P) + y_P$, and $y_R = -m(x_R - x_P) - y_P$.

Theorem 1.1 (Group law)**.** Let $E : y^2 = x^3 + Ax + B$, $P = (x_P, y_P), Q = (x_Q, y_Q)$. Let $R = P \oplus Q = (x_R, y_R)$. Then:

1. If $P, Q \neq \infty, x_P \neq x_Q$:
   - $x_R = m^2 - x_P - x_Q, m = \frac{y_Q - y_P}{x_Q - x_P}$
   - $y_R = -m(x_R - x_P) - y_P$
2. If $P = Q$, but $y_P \neq 0$:
   - $x_R = m^2 - 2x_P, m = \frac{3x_P^2 + A}{2y_P}$
   - $y_R = -m(x_R - x_P) - y_P$
3. If $x_P = x_Q$:
   - $R = \infty$
4. If $P = Q, y_P = 0$:
   - $R = \infty$

5. $\infty + P = P + \infty = P$

Then, the curve E with $\oplus$ forms a group, i.e.:

- If $P, Q \in E \to P \oplus Q \in E$
- ∞ serves as the null element, such that $\infty \oplus P = P \oplus \infty = P$
- There is a negation, i.e. for all P, there is a Q, such that $P \oplus Q = \infty$ (This is true when $P = (x_P, y_P) \to Q = (x_P, -y_P)$)
- $\oplus$ is commutative, i.e. $P \oplus Q = Q \oplus P$
- $\oplus$ is associative, i.e. $P \oplus (Q \oplus R) = (P \oplus Q) \oplus R$

Proof. We will not prove this theorem. □

2 Elliptic curves modulo P

Definition 2.1. Let $p > 3$ be a prime and $E_{A,B} = \{y^2 = x^3 + Ax + B \pmod p\} \cup \infty$ with $4A^3 + 27B^2 \neq 0 \pmod p$

Remark 2.1. Everything works as before, but with modular arithmetic modulo p.

Example: $E : y^2 = x^3 + x + 1 \pmod 5$
We first check whether this is a valid elliptic curve: $4A^3 + 27B^2 = 4 + 27 = 31 \not\equiv 0 \mod 5$

x	$x^3 + x + 1$	y	points
0	1	± 1	$(0,1), (0,4)$
1	3	-	-
2	1	± 1	$(2,1), (2,4)$
3	1	± 1	$(3,1), (3,4)$
4	4	± 2	$(4,2), (4,3)$
∞			

$|E| = 9$
Let's compute $(3,1) \oplus (2,4)$:

$$m = \frac{1-4}{3-2} = -3 = 2 \tag{5}$$
$$x_R = m^2 - 3 - 2 = 4 - 3 - 2 = 4 \tag{6}$$
$$y_R = -m(4-3) - 1 = -2 \cdot 1 - 1 = -3 = 2 \to \tag{7}$$
$$\to (x_R, y_R) = (4,2) \tag{8}$$

Remark 2.2. $E = \{n \cdot (0,1) \cdot n = 0, \dots, 8\}$, where $(0,1) \oplus (0,1) \oplus \cdots \oplus (0,1) = nx$.
$p \cdot (0,1) = \infty = 0 \cdot (0,1) \to E \cong \mathbb{Z}_9$

Theorem 2.1 (Hasse-Weil)**.** Let E be an elliptic curve modulo p. Then: $||E| = (p+1)| \leq 2\sqrt{p}$.

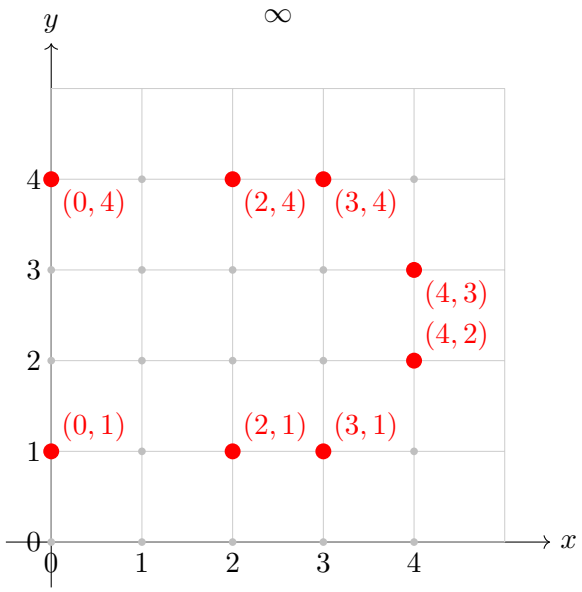


Figure 1: The points of $E : y^2 \equiv x^3 + x + 1 \pmod 5$ in $\mathbb{F}_5^2$, with the point at infinity ∞.

Theorem 2.2. Let E be an elliptic curve modulo p. Then, $E \cong \mathbb{Z}_N$ or $E = \mathbb{Z}_N \times \mathbb{Z}_M$ with $M|N$. That is, either:

- $E = \{n \cdot P, n \in \mathbb{Z}_N\}$ for some $P \in E$ (special case: $|E| = N$), or
- $E = \{n \cdot P + kQ, n \in \mathbb{Z}_N, k \in \mathbb{Z}_M\}$ for some $P, Q \in E$. This is unique (special case: $|E| = N \cdot M$)

Public Key Crypto notes - 11

By contributors

April 27. 2026

1 Projective coordinates

Recall 1.1. $\mathbb{R}^2$ is the two-dimensional affine plane. In this space, there are:

- Points,
- Lines
- $\mathbb{P}(\mathbb{R}^2)$ is the two-dimensional projective space, that contains:
  - affine points,
  - affine lines,
  - a "projective" point: meeting points of parallel lines,
  - a "projective" line: line of all projective points.

Definition 1.1 (Projective points)**.** For $x_1, y_1, z_1, x_2, y_2, z_2$, where $((x_i, y_i, z_i) \neq (0, 0, 0))$, we say $(x_1, y_1, z_1) \sim (x_2, y_2, z_2)$ if $x_2 = \lambda x_1, y_2 = \lambda y_1, z_2 = \lambda z_1$ for some $\lambda \neq 0$.

This $\sim$ is an equivalence relation. Let $(x : y : z)$ be a representative of the equivalence classes of (x, y, z).

Example: $(1 : 1 : 1) = (2 : 2 : 2) = (3 : 3 : 3)$, but $(1 : 1 : 1) \neq (2 : 1 : 0)$.

Remark 1.1. If $z \neq 0$, then $(x : y : z) = (\frac{x}{z} : \frac{y}{z} : 1)$. These will be the affine points.

If $z = 0$, then $(x : y : z) = (x : y : 0)$. These will be the projective points.

Definition 1.2 (Homogenous polynomial)**.** A polynomial $F \in \mathbb{R}[x, y, z]$ of form $F = \Sigma_{c_{i,j,k}} x^i y^j z^k$ is *homogenous* if $c_{i,0,k} \neq 0$, then $i + j + k = d$ for some d. Here, $\deg F = d$.

Examples: $F_1 = c_1x + c_2y + c_3$ is not homogenous, while $F_2 = c_1x + c_2y + c_3z$ is $\deg = 1$ homogenous.

Remark 1.2. If F is homogenous and $(x_1 : y_1 : z_1) = (x_2 : y_2 : z_2)$, then $F(x_1, y_1, z_1) = 0 \iff F(x_2, y_2, z_2) = 0$.

Indeed, if:

$$x_2 = \lambda x_1, \qquad y_2 = \lambda y_1, \qquad z_2 = \lambda z_1. \tag{1}$$

Then

$$F(x_2, y_2, z_2) = F(\lambda x_1, \lambda y_1, \lambda z_1) \tag{2}$$
$$= \sum_{i+j+k=d} c_{i,j,k}(\lambda x_1)^i(\lambda y_1)^j(\lambda z_1)^k \tag{3}$$
$$= \lambda^d \sum_{i+j+k=d} c_{i,j,k} x_1^i y_1^j z_1^k \tag{4}$$
$$= \lambda^d F(x_1, y_1, z_1). \tag{5}$$

Definition 1.3. A line of the projective space is the zero-set of a deg = 1 homogenic polynomial.
Example 1: Take a horizontal line in $\mathbb{R}^2$. $y + c = 0 \rightarrow$ we can homogenize this: $y + cz = 0$.
$(x : y : z)$ is a zero.

- If $z = 1 \rightarrow$ affine point.
- If $z = 0 \rightarrow y = 0 \rightarrow x \neq 0$.

So $(x : 0 : 0) = (1 : 0 : 0)$.
Moreover, $(1 : 0 : 0) = (-1 : 0 : 0)$.
Example 2: Take a vertical line $x+c = 0 \rightarrow$ make it homogenous $\rightarrow x+cz = 0 \rightarrow x+0y+cz = 0$.

- If $z = 1 \rightarrow$ affine points.
- If $z = 0 \rightarrow x = 0 \rightarrow y \neq 0 \rightarrow (0 : y : 0) = (0 : 1 : 0)$.

1.1 Homogenizing

Let $E : y^2 = x^3 + Ax + B$ and $F = x^3 + Ax + B - y^2$. Let's homogenize F:
$H = x^3 + Axz^2 + Bz^3 - y^2z$
The point of E:

- If $z = 1 \rightarrow H(x, y, 1) = 0 \iff (x, y) \in E$.
- If $z = 0 \rightarrow H(x, y, 0) = x^3 \rightarrow x = 0 \rightarrow y \neq 0$.

Thus, $(x : y : 0) = (0 : 1 : 0)$ is the only fully projective point of E.

Remark 1.3.

- A similar addition law can be given, but we do not have to distinguish $P = \infty, P \neq \infty$.
- In implementations, only projective coordinates are used.
- It speeds up computations as modular inversions can be eliminated.

2 Elliptic curve cryptography

Remark 2.1. Let P be a prime, and let E be an elliptic curve over $\mathbb{Z}_p$. Then, the computation $n \to nP$ can be easily computed using the double-and-add algorithm (where n is an integer, and $P \in E$).

i.e. $n = \sum_{i=0}^{\lambda} b_i z^i, b_i \in \{0,1\}$.
$nP = 2(2((\dots(2b_iP)\dots) \oplus b_2P) \oplus b_1P) \oplus b_0P$.
There are $\approx 2l = 2\log_2 n$ many curve operations.
But computing n from P, nP is hard. This is called the DLog problem on the curve.[1]

Techniques for attack:

1. Brute-force: trying all possible n values. The possible values of $n \approx |E| \approx P$ (if P is not trivial).
2. There are smarter algorithms (like baby-step-giant-step) having a running time of approximately $\sqrt{P}$.
3. Index calculus does *not* work.

Hence, to achieve the same security level AES-128, one needs P to be approximately 2^{256} bit size. For RSA and ElGamal encryption, one needs 3000 bits for the same security level.

2.1 Elliptic Curve Diffie-Hellman key exchange

Let P be a prime and E be an elliptic curve over $\mathbb{Z}_P, P \in E$ of order l (i.e. $lP = \infty$, for $0 < k < l, kP + \infty$).

As $E \tilde{=} \mathbb{Z}_N$ or $\mathbb{Z}_N \times \mathbb{Z}_M$, E and P can be chosen such that $l \approx P$.

1. Alice chooses $a \in_R \mathbb{Z}_l$ and sends it to Bob.
2. Bob chooses $b \in_R \mathbb{Z}_l$ and sends it to Alice.
3. $a(bP) = abP = b(aP)$.
4. The shared key is $x = abP$

2.2 Elliptic Curve ElGamal encryption

- Gen:
  - Let P be a prime and E be an elliptic curve over $\mathbb{Z}_P, P \in E$ or order l.
  - $a \in_R \mathbb{Z}_l, Q = aP$.
  - $sk = a, pk = (p, E, P, Q)$.
- Enc:
  - Given $pk = (p, E, P, Q)$ and let $M \in E$ be a message.
  - $c = (rP, M \oplus rQ)$ with $r \in_R \mathbb{Z}_l$.
- Dec:
  - Given $sk = a$, and the ciphertext $c = (R, S)$, the message is $M = S \ominus aR$.

[1] Where previously, we had g^n as the basis of security, we now have nP for the same purpose.

Public Key Crypto notes - 12

By contributors

May 04. 2026

1 Elliptic Curve ElGamal Signature Scheme

Let p be a prime, E be an elliptic curve over $\mathbb{Z}_p$, $P \in E$ of prime order l. Also, let $f : E \to \mathbb{Z}$ (typical choice: $f(x, y) = x$).

- KeyGen:
  - $sk = a \in_R \mathbb{Z}_l$, $pk = (p, E, P, l, a \cdot P = Q)$.
- Sign:
  - Let $m \in \mathbb{Z}_l$,
  - Choose $r \in_R \mathbb{Z}_l, R = rP \in E$,
  - Compute $s = k^{-1}(m - a \cdot f(R))$,
  - Signature: (m, R, s)
- Verify:
  - Let $pk = Q, sign = (m, R, s)$
  - $v_1 = sR + f(R) \cdot Q$
  - $v_2 = m \cdot P$
  - Verify: $v_1 \stackrel{?}{=} v_2$
- Correctness:

$$v_1 = sR + f(R)Q = k^{-1}(m - af(R))kP + f(R)aP = \tag{1}$$
$$= mP - af(R)P + af(R)P = mP = v_2 \tag{2}$$

2 Elliptic Curve Digital Signature Algorithm (ECDSA)

- Standardized by NIST.
- KeyGen:
  - $sk = a \in_R \mathbb{Z}_l, pk = Q = aP \in E$
- Sign:
  - Let $m \in \mathbb{Z}_l$ be the message.

- Choose $k \in_R \mathbb{Z}_l, R = kP \in E$.
- Compute $s = k^{-1}(m + af(R))$
- Signature: (m, R, s)

- Verify:
  - $u_1 = s^{-1} \cdot m \in \mathbb{Z}_l$
  - $u_2 = s^{-1} \cdot f(R) \in \mathbb{Z}_l$
  - $v = u_1 P + u_2 Q$

- Correctness:

$$\begin{aligned} v &= u_1 P + u_2 Q && (3)\\ &= s^{-1} \cdot mP + s^{-1} \cdot f(R)a \cdot P && (4)\\ &= (s^{-1} \cdot m + s^{-1} \cdot f(R)a)P && (5)\\ &= k(m + af(R))^{-1}(m + af(R)) \cdot P && (6)\\ &= kP = R && (7) \end{aligned}$$

Remark 2.1 (Computational costs of Verify)**.**
EC ElGamal: 3 scalar multiplications, 1 addition.
ECDSA: 2 scalar multiplications, 1 addition on E.
ECDSA is much faster.

Remark 2.2. One can consider elliptic curves over finite field $\mathbb{F}_q$ (q is a prime power) instead of $\mathbb{Z}_p$.

However, if $q = 2^n$ or $q = 3^n$, we need a different curve equation.

3 Pairing-based crypto

Let E be an elliptic curve over $\mathbb{Z}_p$, and for $n||E|$, let $E[n] = \{P \in E, nP = \infty\}$.

If n is large enough, then $E[n]$ can be almost as large as E, e.g. $|E| = l$ is a prime $E[l] = E$.

Definition 3.1 (Pairing)**.** A pairing is a map $e : E[n] \times E[n] \to \mathbb{Z}_p$ with:

1. e is bilinear. $e(aP, Q) = e(P, aQ) = e(P, Q)^a$
2. e is non-trivial. If $e(P, Q) = 1$ for all Q or $e(Q, P) = P$ for all $Q \to P = \infty$

Remark 3.1. For appropriate choice of parameters (e.g. $n|p-1$), there are curves E over $\mathbb{Z}_p$ such that a pairing exists.

Having pairing, one can perform cryptographic attacks.

1. If there is a pairing, then the Decisional Diffie-Hellman problem is easy.
   - Given $aP, bP, Q \to$ decide whether $abP \stackrel{?}{=} Q$
   - Idea: $e(aP, bP) = e(P, abP) \stackrel{?}{=} e(P, Q)$
2. If one can solve DLog in $\mathbb{Z}_p$, then they can do it on E too.

- Given $P, aP = Q \to a =?$
- Idea: $e(P,Q) = e(P,aP) = e(P,P)^a$. If $g = e(P,P) \in \mathbb{Z}_p$ given $e(P,Q) = g^a \in \mathbb{Z}_p$.
- So p has to be the same size as in classical (not elliptic curve) crypto, i.e. $p \approx 3000-4000$ bits.

3.1 Protocol 1

3-party Diffie-Hellman key exchange protocol.

- Person A: $a \in_R \mathbb{Z}_n$, sends $s_A P$.
- Person B: $b \in_R \mathbb{Z}_n$, sends $s_B P$.
- Person C: $c \in_R \mathbb{Z}_n$, sends $s_C P$.
- Then $e(bP, cP)^a = e(aP, cP)^b, e(aP, bP)^c = e(P,P)^{abc}$.

3.2 Protocol 2 - Identity-based encryption

Goal: Design a PKE such that the public key is an identifier (ID).

Definition 3.2 (Boneh-Franklin IBE)**.**

- A, B users
- TA: Trusted Authority
- Protocols:
  - Setup,
  - Extraction,
  - Encryption,
  - Decryption
- Setup:
  - TA chooses:
    - Primes p, l, an elliptic curve E over $\mathbb{Z}_p$ such that $\exists e : E[n] \times E[n] \to \mathbb{Z}_p$ and a point $P \in E[n]$ of order l.
    - $H : \{0,1\}^* \to P \in E[l]$
    - $s \in_R \mathbb{Z}_l, P_{pub} = sP$, where s is secret, P_{pub} is public.
- Extraction:
  - Let Bob's ID be ID_B.
  - TA computes: $Q_B = H(ID_B), D_B = sQ_B$.
  - Bob's $sk = D_B$
- Encryption:
  - Let ID_B be the ID of Bob.

- A computes: $Q_B = H(ID_B), g_B = e(Q_B, P_{pub})$
- $r \in_R \mathbb{Z}_l$, cipher text: $c = (rP \in E, m \cdot g_B^r \in \mathbb{Z}_l)$ on message m.

- Decryption:
  - B is given $c = (U, v)$ having $D_B : h_B = e(D_B, U)$
  - $m = \frac{v}{h_B}$

Correctness:

$$\begin{aligned} h_B &= e(D_B, U) = && (8)\\ &= e(sQ_B, rP) = && (9)\\ &= r(Q_B, sP)^r = && (10)\\ &= g_B^r && (11) \end{aligned}$$

And:

$$\frac{v}{h_B} = \frac{m \cdot g_B^r}{g_B^r} = m$$